

Dynamic analysis of Android apps with Octopus

Module 4 of 4

3 hours

Defensive Lab Agency | pts-project.org

Main objective

Run a suspicious Android app under observation, record everything it does, and turn the recording into evidence and detection rules in Colander.

You will learn to set up a safe analysis device, instrument an app with **Octopus**, decrypt its TLS traffic without breaking it, attribute each transmission to the code that sent it, and write a report.

Where we left case LAB-001

- beaconing to `sync.update-wire.<lab domain>` seen on the network
- the app found by MVT: a **fake Wire**, signed by "Zeta", with an injected `org.xmlpush.v3` package
- static analysis hits a wall: **241 obfuscated classes, encrypted strings**

Open questions for today:

- what does the injected code do when it runs?
- what data does it touch, and where does it try to send it?

Agenda

1. Static vs dynamic analysis, and safety (15 min)
2. How Octopus works (15 min)
3. Setting up an analysis device (15 min)
4. Instrumenting an app: a known app first (25 min)
5. From recording to Colander: decrypt, attribute, decode (30 min)
6. Detection rules and report (15 min)
7. The fake Wire under Octopus (20 min)
8. Exercise, wrap-up (45 min)

Static vs dynamic analysis

	Static	Dynamic
What	read the app without running it	run the app and observe it
Tools	jadx, Pithus, droidlysis	Octopus, PiRogue intercept
Shows	permissions, certificate, code, strings	real traffic, decrypted data, what code sent what
Blocked by	obfuscation, encrypted strings, code downloaded later	anti-analysis tricks, features that do not trigger
Risk	low	the app runs: it can reach its operator

They complement each other. Our case: static told us *who signed it*; dynamic will tell us *what it does*.

Safety first

Running a sample means **letting malware execute**. Before you do:

- a **dedicated** rooted phone or an emulator: no SIM, no accounts, no personal data, wiped after
- a dedicated network: the app may contact its operator, who then sees **your IP**
- check the case's **PAP**: running a sample is an *active* action
- record what you did and when: the run is part of the evidence
- never on the victim's phone, never on your work laptop's host OS

If the case is sensitive, ask for help. Dynamic analysis is where mistakes cost the most.

Question

You received a suspicious APK from a human rights lawyer. The case is TLP:RED, PAP:RED.

Can you run it in Octopus on the office Wi-Fi? What are your options?

What Octopus is

A dynamic analysis framework for Android apps, part of PTS. It instruments apps with **Frida** and records:

- **screen**: video of the session (`screen.mp4` , 3 minutes max)
- **network**: full capture on the device (`traffic.pcap`)
- **TLS keys**: extracted from memory with friTap (`sslkeylog.txt`)
- **sockets**: every connect, read, write, with its **stack trace**
- **crypto**: AES and RSA operations, keys, cleartext (`aes_info.json`)
- **API hooks**: advertising IDs, device information

Runs on your computer (or your PiRogue), talks to the phone through ADB, over USB or the network.

Decrypting TLS without breaking it

MITM proxy (e.g. mitmproxy): impersonates the server with a fake certificate.

- needs the device to trust your CA
- **fails against certificate pinning**, and the app may notice

Key extraction (Octopus, friTap): reads the TLS keys from the app's memory while it runs.

- the traffic is **not modified**, the app behaves normally
- works even with certificate pinning
- needs a **rooted** device

For evidence, not altering what you observe matters.

Setting up: what you need

- a computer with **Python 3.11+**, `pipx` and **ADB**
- a **rooted** Android device with USB debugging, or a rooted **emulator**

```
pipx install pirogue-octopus
octopus device list          # "0 device(s) found"? cable, unlock, accept the prompt
```

Physical phone: most realistic, some malware detects emulators.

Emulator: free, disposable, snapshot and reset between runs.

A rooted emulator in four commands

Use a **Google APIs** image, not a Google Play one (those are much harder to root).

```
sdkmanager "system-images;android-33;google_apis;x86_64"      # arm64-v8a on Apple Silicon
avdmanager create avd -n octopus -k "system-images;android-33;google_apis;x86_64" -d pixel_c
emulator -avd octopus -no-snapshot-load
adb root && adb shell whoami                                     # → root
```

Our fake Wire ships native code for `arm64-v8a`, `armeabi-v7a`, `x86` and `x86_64`: it runs on any emulator.

Running an analysis

1. install the app: `adb install sample.apk`, and **do not launch it**
2. make sure it is not running (Settings → Apps → Force stop)
3. start Octopus:

```
octopus instrument usb --output-path ./lab001_run1 --duration 180
```

4. when Octopus says **Waiting for data**, launch the app and use it
5. `Ctrl+C` (or wait for `--duration`) to stop

Octopus instruments processes **when they start**: an app already running is not instrumented.

Useful options

Option	Why
<code>--duration 180</code>	stop automatically, match the 3-minute screen recording
<code>-o ./lab001_run1</code>	one folder per run: name it after the case and the run
<code>-ns</code>	no screen recording, e.g. when the screen shows personal data
<code>-nn</code>	no network capture
<code>tcp --device-host <IP></code>	device over the network instead of USB
<code>--adb-host <IP></code>	use an ADB server on another machine

Live demo, part 1

Instrumenting a known app

What Octopus records

File	Content
<code>traffic.pcap</code>	full network capture
<code>sslkeylog.txt</code>	TLS keys to decrypt the capture
<code>socket_trace.json</code>	every socket operation, with the stack trace
<code>aes_info.json</code>	encryption operations, keys, cleartext
<code>ad_ids.txt</code>	advertising IDs seen
<code>device.json</code>	device properties: model, IMEI, Android version
<code>screen.mp4</code>	screen video (3 min max)
<code>experiment.json</code>	timings of the session, list of files

Hash the folder's files and store them in the case: this run is evidence.

Reading the traffic yourself

Inject the keys into the capture, open it in Wireshark:

```
editcap --inject-secrets tls,sslkeylog.txt traffic.pcap decrypted.pcapng
```

Or convert it to a HAR file with **pcapng-utils**: open it in any browser's network inspector, with the stack trace attached to each request.

Always check the PCAP in Wireshark too: Colander only displays **HTTP** traffic.

From Octopus to Colander

Colander treats a run as a **PiRogue experiment**: the files described by `experiment.json`.

```
# on your PiRogue, with the Colander connector configured (once):
pirogue-colander config -u "<Colander URL>" -k "<your API key>"
# copy the Octopus output folder to the PiRogue, then:
pirogue-colander collect-experiment -c "<case ID>" -t sample.apk ./lab001_run1
```

The exact command, with your case ID, is shown at the bottom of **Collect** → **PiRogue experiment** in Colander.

Alternative on the PiRogue itself: `sudo pirogue-intercept-gated -o <output>` records the same kind of experiment.

Decrypt: what Colander does for you

In the experiment, click **Decrypt**. In the background, Colander:

1. decrypts `traffic.pcap` with `sslkeylog.txt`
2. attaches to each flow the **stack trace** that sent it (Community ID + timing)
3. runs **Exodus Privacy** rules on the stack: which **SDK** sent the data
4. replaces payloads encrypted *inside* TLS with their **cleartext** from `aes_info.json`
5. adds **geoip**: country, organization, ASN of the server

Reading a transmission

For each request, Colander shows:

- direction, protocol, size, source and destination
- the **SDK** detected, and its purpose (analytics, advertisement...)
- the **detection rules** that matched
- the Android **process**, thread and socket
- HTTP headers
- the **stack trace**: which part of the app sent it
- the payload: raw, decrypted, decoded

The stack trace is the key question for our case: is it `com.waz...` (Wire's code) or `org.xmlpush.v3` (the injected code)?

Decoding payloads

Not everything is readable JSON. For each payload:

- **Send to CyberChef** (raw hex or decrypted data)
- build a recipe until it is readable (Base64, gzip, protobuf...)
- **Import decoded content** back into Colander

Detection rules run on the decoded content, so decode before you detect.

Still unreadable? Check `aes_info.json`: the app may encrypt before sending.

Detection rules

Collect → **Detection rules**, type Yara. They are applied to the URL, the HTTP headers and the decoded payload.

```
rule lab001_device_model {  
  strings: $s = "sdk_gphone64" nocase  
  condition: $s  
}  
rule lab001_c2_domain {  
  strings: $s = "update-wire." nocase  
  condition: $s  
}
```

In the experiment, click **Detect**. Changed a rule? Click **Detect** again: rules are not re-applied automatically.

The report

In the experiment, **View report**, then print to PDF. It contains:

- the artifacts (PCAP, screen recording...) and their digital signatures
- the detection rules and the detection summary
- every transmission: destination, organization, the code that sent it
- the data identified: advertising ID, location, device identifiers...
- the purpose of each recipient, from Exodus Privacy's tracker classification

Anyone can verify it: the PCAP and the key log open in Wireshark.

Live demo, part 2

The known app in Colander

The fake Wire under Octopus: what to look for

- **screen.mp4**: which permissions does it ask for, and in which order?
- **DNS and connections** to anything that is not Wire's infrastructure
- **socket traces** with frames in `org.xmlpush.v3`: traffic from the injected code
- **aes_info.json**: keys and cleartext, maybe its decrypted configuration
- **device.json vs payloads**: does it send the IMEI, the model, the accounts?

A dead C2 is still a finding: **failed connections and DNS lookups** give you domains, IPs and timing.

Live demo, part 3

Running the fake Wire

Case LAB-001: closing

Question	Answer	Source
Is the device compromised?	Yes	network + MVT
What is the implant?	fake Wire, <code>org.xmlpush.v3</code> injected	AndroidQF, jadx
What does it do?	the behavior you recorded: permissions used, data touched, destinations	Octopus run
Which code sends what?	stack traces into <code>org.xmlpush.v3</code>	Colander experiment
How do we detect it next time?	Suricata rules, Yara rules, IOCs in a feed	PiRogue, Colander

The case is ready for a report, and for a feed to partners.

Limitations to know

- not every TLS library is supported by the key extraction
- Colander only displays **HTTP**: check other protocols in Wireshark
- the screen recording stops after **3 minutes**
- some malware detects **root** or **emulators** and behaves differently
- a feature that is not triggered is not recorded: **no traffic ≠ harmless**
- rules must be re-applied after each change

When the results do not add up, ask for help rather than conclude.

Exercise

In pairs, with an emulator or a test phone:

1. pick an app from the list the facilitator gives you
2. instrument it with Octopus for 3 minutes, using it like a normal user would
3. find in the output: the advertising ID, and one tracker domain
4. bring it into a Colander practice case, **Decrypt**
5. name the SDKs that received data, and for what purpose
6. write one Yara rule that proves the device model is sent, and **Detect**
7. generate the report

40 minutes, then each pair presents one surprising finding.

Wrap-up: the whole curriculum

- **Module 1:** PiRogues and Colander deployed, users, teams, fleet
- **Module 2:** capture, triage, IOCs, TLP and PAP
- **Module 3:** remote capture, AndroidQF + MVT, evidence, sharing
- **Module 4:** dynamic analysis, decryption, attribution, detection rules, report

One case, followed from an SMS to a report and a feed.

Questions & resources

- Octopus: [🔗 docs.pts-project.org/docs/Octopus/overview](https://docs.pts-project.org/docs/Octopus/overview)
- Guide, analyze app traffic with Colander: [🔗 docs.pts-project.org/guides/analyze-traffic-with-colander](https://docs.pts-project.org/guides/analyze-traffic-with-colander)
- Guide, static analysis with jadx: [🔗 docs.pts-project.org/guides/static-analysis-with-jadx](https://docs.pts-project.org/guides/static-analysis-with-jadx)
- Exodus Privacy reports: [🔗 reports.exodus-privacy.eu.org](https://reports.exodus-privacy.eu.org)
- Community Discord: [🔗 discord.gg/qGX73GYNdp](https://discord.gg/qGX73GYNdp)

Thank you!